FIAP · CIBERSEGURANÇA

Football Career API

API REST de modo carreira de futebol com simulação de partidas e banco de dados SQLite

Documentação técnica, arquitetura em nuvem, banco de dados, endpoints, implantação e revisão do código.

Aluno	Kauã do Sacramento Soares
Turma	FIAP — 1TDCPA
Professor	Eduardo Popovici
Disciplina	[preencher]
Data	Novembro de 2026
Repositório	[inserir link real do GitHub]
URL da API	[inserir URL pública após validar o deploy]

1. Introdução

Este documento descreve a implementação recebida da Football Career API, uma API REST que simula um modo carreira de futebol. O projeto permite criar carreiras independentes em dois formatos — técnico (MANAGER) e jogador (PLAYER) —, administrar clubes, disputar temporadas, registrar resultados e armazenar estatísticas em SQLite.

A solução foi desenvolvida em Python com FastAPI e utiliza SQLite como banco de dados em arquivo. A versão revisada inclui chave de API opcional por variável de ambiente, testes automatizados e rotina de backup. O roteiro de implantação utiliza uma máquina virtual Linux no Microsoft Azure, Nginx como proxy reverso e systemd para manter a API em execução.

O objetivo da documentação é explicar a aplicação, suas regras, arquitetura, banco de dados, endpoints, implantação acadêmica, testes e evidências necessárias para demonstrar o funcionamento de ponta a ponta.

> Escopo e evidências: os testes automatizados desta versão foram executados localmente. Ainda não foram fornecidos URL real do GitHub, endereço público da API, exportação dos recursos Azure ou capturas do ambiente implantado. Esses campos permanecem para preenchimento com evidências reais.

1.1. Requisitos do trabalho

Requisito	Como o projeto atende	Onde documentar
API básica com banco SQL	FastAPI com persistência SQLite no arquivo career.db.	Seções 5, 6 e 7
Aplicação funcional	Criação e consulta de carreiras, partidas, temporadas, mercado e estatísticas.	Seções 2, 7 e 9
Hospedagem de ponta a ponta	README descreve VM Ubuntu, Nginx, systemd e acesso pela porta 80. Deploy real ainda precisa de evidência.	Seções 3, 4 e 8
Repositório GitHub	Arquivos main.py, requirements.txt, README.md e .gitignore. Link real a inserir.	Seção 10
Esquemas visuais	Arquitetura, modelo de dados e fluxo de uma carreira.	Figuras 1, 2 e 3
Prints de tela	Locais reservados para evidências do Azure, terminal, Swagger e GitHub.	Seções 8 e 9
Elementos de nuvem	VM/SKU, disco, IP, rede, NSG e região precisam ser confirmados no portal.	Seção 4
Linguagem e tecnologias	Python, FastAPI, Uvicorn, SQLite, Nginx, systemd e Git.	Seção 5

2. A aplicação: modo carreira de futebol

A API cria um universo separado para cada carreira. Ao iniciar uma nova carreira, o sistema gera oito clubes fictícios, seus elencos, dez jogadores livres e o calendário da temporada. Cada campeonato tem turno e returno, totalizando 14 rodadas.

Elemento	Regra implementada
Clubes	8 clubes fictícios, cada um associado a uma cidade e a uma força inicial.
Elencos	18 jogadores por clube no início: 2 GK, 6 DEF, 6 MID e 4 ATT. O modo jogador adiciona o atleta criado ao elenco do clube escolhido.
Agentes livres	10 jogadores sem clube, disponíveis para o mercado.
Posições	GK (goleiro), DEF (defensor), MID (meio-campista) e ATT (atacante).
Atributos	Idade, rating, potencial, titularidade e situação de aposentadoria.
Formações	4-4-2, 4-3-3 e 3-5-2.

Elemento	Regra implementada
Táticas	BALANCED, ATTACK e DEFENSE.
Temporada	14 rodadas, com partidas simuladas para todos os clubes.

2.1. Simulação das partidas

A API seleciona os 11 titulares e calcula o poder ofensivo e defensivo de cada equipe considerando posição, rating, formação e tática. O mandante recebe um bônus. A expectativa de gols é limitada a uma faixa entre 0,15 e 4,5 e os gols são sorteados por uma distribuição de Poisson.

- O peso de cada jogador como autor de gol depende da posição e do rating. Atacantes têm maior peso; goleiros, peso muito baixo.
- Existe 70% de chance de uma assistência para cada gol. O assistente é escolhido entre os jogadores de linha que não marcaram aquele gol.
- Cada titular recebe uma nota de partida entre 3,0 e 10,0, influenciada pelo resultado, gols, assistências e desempenho defensivo.
- Os eventos de gol são ordenados pelo minuto antes de serem retornados na resposta da API.

2.2. Modo técnico (MANAGER)

No modo técnico, o usuário escolhe um dos oito clubes e passa a administrar sua equipe. A carreira acompanha a confiança da diretoria, o orçamento, a escalação e as movimentações no mercado.

Recurso	Funcionamento
Confiança da diretoria	Começa em 60. Vitória acrescenta 4 pontos; derrota retira 4; empate não altera a confiança. A confiança é limitada entre 0 e 100.
Demissão	A carreira é encerrada quando a confiança chega a 15 ou menos. No fim da temporada, a posição final também ajusta a confiança em relação à meta.
Meta esportiva	A posição-alvo inicial é calculada pela força relativa do clube entre os oito times.
Orçamento	Começa em 30 milhões. A receita de fim de temporada varia conforme a posição final.
Escalação	Permite escolher formação, tática e exatamente 11 jogadores, respeitando as posições exigidas pela formação.
Mercado	Permite consultar jogadores, filtrar por posição/preço e comprar ou vender durante a janela de transferências.
Janela	Abre antes da rodada 1 e antes da rodada 8.

2.3. Modo jogador (PLAYER)

No modo jogador, o usuário cria um atleta com nome, posição e idade entre 16 e 21 anos. O atleta começa como titular no clube de menor força e passa a registrar partidas, gols, assistências, notas médias, rating e potencial.

- A evolução ocorre ao fim da temporada. Atletas mais jovens tendem a evoluir; jogadores mais velhos podem perder rating. O desempenho do atleta do usuário concede bônus ou penalidade adicional.
- No início das temporadas seguintes, a API pode gerar até três propostas de outros clubes, classificadas como TITULAR ou RESERVA.
- Ao aceitar uma proposta, o jogador muda de clube e as demais propostas pendentes da temporada são marcadas como rejeitadas.
- O usuário pode encerrar a carreira manualmente. A aposentadoria automática ocorre quando o atleta chega aos 36 anos.

Fluxo principal de uma carreira

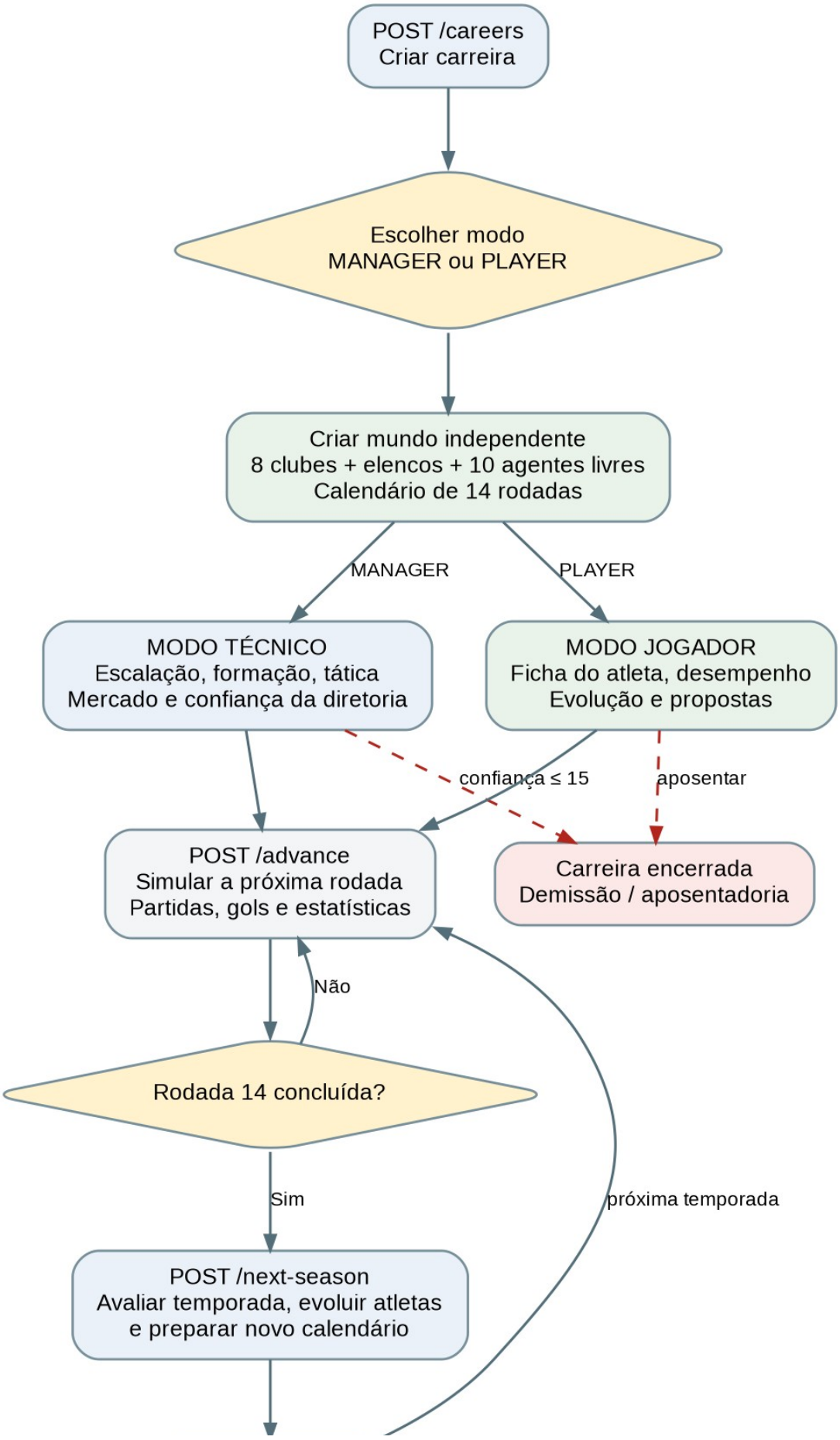


Figura 1 — Fluxo de criação, simulação e progressão de uma carreira.

3. Arquitetura da solução

O roteiro de implantação utiliza uma máquina virtual Linux no Azure. O cliente acessa o IP público da VM por HTTP; o Network Security Group (NSG) filtra o tráfego; o Nginx recebe as requisições na porta 80 e encaminha o tráfego para o Uvicorn/FastAPI, que executa as regras da aplicação e consulta o banco SQLite armazenado no disco da máquina.

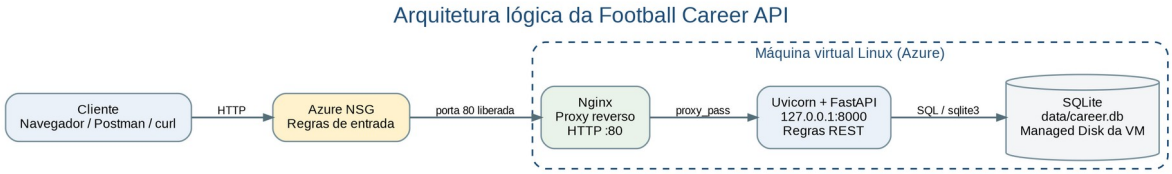


Figura 2 — Arquitetura lógica: cliente HTTP, NSG, Nginx, FastAPI e banco SQLite.

3.1. Fluxo de uma requisição

1. O usuário acessa uma rota, por exemplo http://<IP_PUBLICO>/careers, pelo navegador, Postman ou curl.
2. O NSG verifica se a porta de destino está liberada e encaminha o tráfego à interface de rede da VM.
3. O Nginx recebe a requisição na porta 80 e a encaminha para 127.0.0.1:8000 por meio de proxy_pass.
4. O Uvicorn entrega a requisição à aplicação FastAPI. Os modelos Pydantic validam os dados recebidos.
5. A aplicação executa comandos SQL no SQLite por meio do módulo sqlite3 e retorna os dados em JSON.

A documentação interativa do FastAPI fica disponível em /docs. Ela permite visualizar as rotas e executar chamadas diretamente pelo navegador. A documentação OpenAPI também pode ser consultada em /openapi.json.

4. Elementos de nuvem (Microsoft Azure)

O README apresenta o Azure como ambiente de hospedagem e descreve uma VM Ubuntu 22.04, com Nginx e systemd. A documentação anterior enviada pelo aluno utilizava uma VM pequena da série B e um Managed Disk, mas os arquivos desta versão não comprovam que a mesma SKU, região ou tamanho de disco foram efetivamente usados.

Recurso Azure	Configuração de referência / campo a confirmar	Função
Assinatura	Azure for Students [confirmar]	Assinatura usada para provisionar os recursos.
Resource Group	[nome real do Resource Group]	Agrupa VM, rede, IP, disco e regras de segurança.
Região	[confirmar no portal]	Localização física dos recursos Azure.
Máquina virtual	Ubuntu Server 22.04 LTS (README)	Executa Nginx, Uvicorn/FastAPI e SQLite.
SKU da VM	[confirmar: ex. Standard_B1s]	Define vCPU, memória e capacidade de processamento.
OS Disk	[confirmar tipo e tamanho]	Armazena o sistema operacional, código e arquivo career.db.
VNet / NIC	[confirmar nomes e faixa de rede]	Conecta a VM à rede virtual e à interface de rede.
IP público	[estático ou dinâmico; inserir IP]	Endereço de acesso externo à API.
NSG	[nome do NSG]	Filtra o tráfego de entrada e saída da VM.
Portas de entrada	22 SSH e 80 HTTP	SSH para administração; HTTP para demonstração da API.
Acesso administrativo	Chave SSH (recomendado)	Evita autenticação administrativa por senha.

Para a demonstração acadêmica, a API é acessada por HTTP na porta 80. Restrinja a porta 22 (SSH) ao IP administrativo e não exponha a porta 8000 do Uvicorn diretamente à internet.

4.1. SKU da máquina virtual

SKU (Stock Keeping Unit) é o identificador do tamanho da VM. Ele determina recursos como vCPU, memória RAM e desempenho de rede/disco. Para uma API acadêmica com baixo volume de requisições, uma VM pequena pode ser suficiente, desde que a memória e o armazenamento comportem o sistema operacional, o ambiente Python e o arquivo do banco.

Característica	Valor a registrar
SKU efetivamente criada	[preencher com o valor exibido no portal Azure]
vCPU	[preencher conforme a SKU]
Memória RAM	[preencher conforme a SKU]
Sistema operacional	Ubuntu Server 22.04 LTS, conforme README
Justificativa	Carga de demonstração, API leve e banco SQLite local.

4.2. Armazenamento e persistência

O banco é SQLite e fica em um único arquivo. No desenvolvimento local, o caminho padrão é data/career.db. A aplicação aceita a variável de ambiente DB_PATH para apontar para outro local. No roteiro de implantação do README, o caminho utilizado é /opt/football-api/data/career.db.

- O arquivo permanece no disco da VM e não exige um servidor de banco de dados separado.
- O diretório data/ e arquivos *.db são excluídos do Git pelo .gitignore, evitando versionar o banco local.
- O script backup_db.py utiliza a API online de backup do SQLite, verifica PRAGMA integrity_check e aplica retenção padrão de 14 dias.
- O backup pode ser agendado com cron; a restauração exige parar o serviço, substituir o arquivo e validar a inicialização.

5. Linguagem e tecnologias

Tecnologia	Uso no projeto	Observação
Python 3	Linguagem da aplicação e regras do jogo.	O código usa bibliotecas padrão como sqlite3, random, math e os.
FastAPI 0.115.6	Framework para disponibilizar os endpoints REST.	Versão fixada em requirements.txt.
Uvicorn 0.32.1	Servidor ASGI que executa a aplicação FastAPI.	Versão fixada em requirements.txt.
Pydantic	Validação dos corpos JSON e dos parâmetros.	Usado por meio dos modelos BaseModel e Field.
SQLite / sqlite3	Banco relacional em arquivo.	Módulo da biblioteca padrão do Python.
Nginx	Proxy reverso diante do Uvicorn.	Configuração do README recebe tráfego HTTP na porta 80.
systemd	Gerenciamento do processo da API na VM.	Permite habilitar o serviço na inicialização e reiniciá-lo em falhas.
pytest + httpx	Testes automatizados dos endpoints e das regras de negócio.	Dependências de desenvolvimento em requirements-dev.txt.
backup_db.py	Cópia consistente e verificação do SQLite.	Script Python com API online de backup e retenção.
Git / GitHub	Versionamento e distribuição do código.	Link do repositório ainda precisa ser inserido.

Dependências declaradas no arquivo requirements.txt:

```
requirements.txt
fastapi==0.115.6
uvicorn[standard]==0.32.1

requirements-dev.txt
```

```
-r requirements.txt
httpx==0.28.1
pytest==8.3.4
```

O projeto entregue é o backend REST; a interface interativa disponível para demonstração é o Swagger em /docs. Ainda não há frontend gráfico, Dockerfile, pipeline CI/CD ou migrações versionadas do esquema SQLite.

6. Modelagem do banco de dados

O banco possui sete tabelas. Cada carreira mantém seus clubes, jogadores, calendário, gols, estatísticas e propostas. A coluna career_id é utilizada para separar os mundos de jogo e evitar que os dados de uma carreira sejam misturados aos de outra.

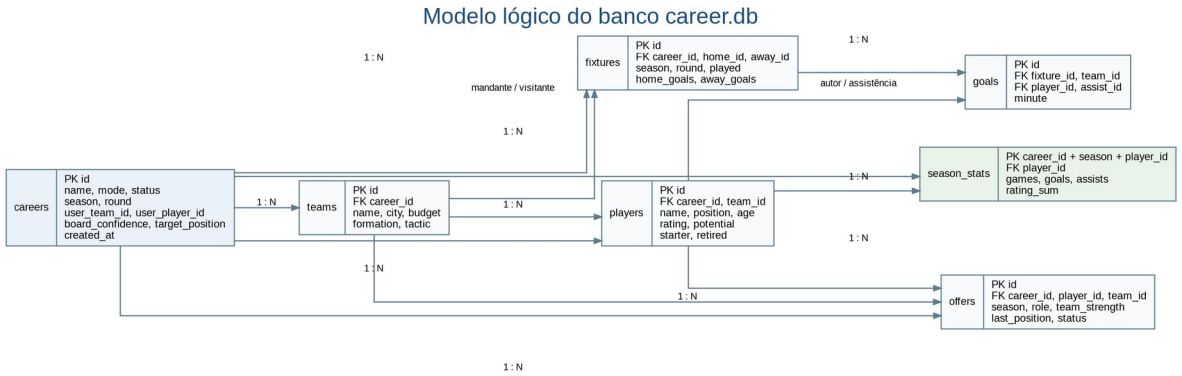


Figura 3 — Modelo lógico do banco SQLite, com as principais chaves e relacionamentos.

Tabela	Finalidade e campos principais
careers	Cadastro da carreira: nome, modo MANAGER/PLAYER, status, temporada, rodada, clube e jogador do usuário, confiança da diretoria e posição-alvo.
teams	Clubes da carreira: nome, cidade, orçamento, formação, tática e vínculo com career_id.
players	Jogadores: clube, nome, posição, idade, rating, potencial, titularidade e aposentadoria.
fixtures	Calendário e partidas: temporada, rodada, mandante, visitante, situação e placar.
goals	Eventos de gol: partida, equipe, autor, assistente e minuto.
season_stats	Estatísticas por temporada e jogador: partidas, gols, assistências e soma das notas. Chave primária composta por career_id, season e player_id.
offers	Propostas de transferência do modo jogador: clube, atleta, papel, força do time, posição anterior e status.

6.1. Integridade e restrições

- A aplicação ativa PRAGMA foreign_keys = ON em cada conexão SQLite.
- A tabela careers restringe mode a MANAGER ou PLAYER e status a ACTIVE, FIRED ou RETIRED.
- A tabela players restringe position aos valores GK, DEF, MID e ATT, e rating/potential aos valores entre 1 e 99 no banco.
- A tabela teams restringe tactic a BALANCED, ATTACK ou DEFENSE.
- A tabela offers restringe role a TITULAR ou RESERVA e status a PENDING, ACCEPTED, REJECTED ou EXPIRED.

Embora a estrutura SQL aceite rating entre 1 e 99, a função de inserção limita ratings criados pela aplicação à faixa 30–95. A faixa real observada depende das regras de geração e evolução do jogo.

7. Endpoints da API

As rotas são agrupadas por tags no FastAPI. O identificador {cid} representa o ID da carreira. As requisições e respostas trafegam em JSON, exceto quando a rota não possui corpo.

Método	Rota	Descrição
GET	/health	Verifica se a API está ativa.
GET	/clubs	Lista os oito clubes disponíveis e suas cidades/forças iniciais.
POST	/careers	Cria uma carreira no modo MANAGER ou PLAYER.
GET	/careers	Lista as carreiras cadastradas.
GET	/careers/{cid}	Retorna o resumo da carreira e dados do clube/jogador.
DELETE	/careers/{cid}	Exclui uma carreira e seus dados relacionados.
GET	/careers/{cid}/standings	Consulta a classificação da temporada.
GET	/careers/{cid}/fixtures	Lista o calendário e os resultados; aceita filtros.
GET	/careers/{cid}/teams	Lista os clubes da carreira.
GET	/careers/{cid}/teams/{team_id}	Detalha um clube e seu elenco.
GET	/careers/{cid}/top-scorers	Consulta artilharia, assistências, jogos e nota média.
GET	/careers/{cid}/squad	Consulta o elenco do clube do usuário e estatísticas.
POST	/careers/{cid}/advance	Simula uma ou mais rodadas.
POST	/careers/{cid}/next-season	Encerra a temporada e prepara a próxima.
PUT	/careers/{cid}/lineup	Modo técnico: define formação, tática e 11 titulares.
GET	/careers/{cid}/market	Modo técnico: lista jogadores disponíveis para transferência.
POST	/careers/{cid}/market/buy	Modo técnico: compra um jogador.
POST	/careers/{cid}/market/sell	Modo técnico: vende um jogador.
GET	/careers/{cid}/player	Modo jogador: ficha, histórico e totais da carreira.
GET	/careers/{cid}/offers	Modo jogador: lista propostas recebidas.
POST	/careers/{cid}/offers/{offer_id}/accept	Modo jogador: aceita uma proposta.
POST	/careers/{cid}/retire	Modo jogador: encerra a carreira manualmente.

7.1. Acesso no escopo acadêmico

A API não implementa autenticação de usuários. As rotas permanecem abertas para que o professor possa testar as funcionalidades diretamente pelo Swagger, Postman ou curl. Essa decisão mantém o projeto simples e adequado à demonstração acadêmica. Como consequência, qualquer pessoa que consiga acessar a API poderá executar as operações expostas; por isso, a aplicação deve ser utilizada somente com dados fictícios e no ambiente controlado da atividade.

O acesso externo de demonstração utiliza HTTP pela porta 80. Não foram configurados domínio, certificado TLS ou HTTPS, pois esses itens não fazem parte do escopo acadêmico definido para este projeto.

7.2. Modelos de entrada

Criação de uma carreira de técnico:

```
POST /careers
Content-Type: application/json

{
 "name": "Minha carreira",
 "mode": "MANAGER",
 "club": "Tigres do ABC"
}
```

Criação de uma carreira de jogador:

```
POST /careers
```

```
Content-Type: application/json

{
 "name": "Minha carreira como jogador",
 "mode": "PLAYER",
 "player": {
  "name": "Kauã Soares",
  "position": "ATT",
  "age": 18
 }
}
```

Alteração de escalação no modo técnico:

```
PUT /careers/{cid}/lineup

{
 "formation": "4-3-3",
 "tactic": "ATTACK",
 "starters": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
}
```

O campo starters é opcional. Quando omitido, a API escolhe automaticamente os melhores jogadores disponíveis para a formação selecionada. Quando informado, deve conter exatamente 11 IDs diferentes e respeitar a distribuição de posições exigida pela formação.

7.3. Parâmetros de consulta

Rota	Parâmetros relevantes
GET /careers/{cid}/standings	season: temporada desejada.
GET /careers/{cid}/fixtures	round: rodada; mine: somente jogos do clube do usuário; season: temporada.
GET /careers/{cid}/top-scorers	season: temporada; limit: quantidade de atletas (1 a 50).
POST /careers/{cid}/advance	rounds: quantidade de rodadas simuladas (1 a 14).
GET /careers/{cid}/market	position: filtro de posição; max_price: teto de preço; limit: quantidade de resultados (1 a 100).

7.4. Fluxo de partida e temporada

```
POST /careers/{cid}/advance
POST /careers/{cid}/advance?rounds=13
GET  /careers/{cid}/standings
GET  /careers/{cid}/top-scorers
POST /careers/{cid}/next-season
```

A chamada advance simula as próximas rodadas, respeitando o limite de 14 rodadas por temporada. Quando a rodada 14 termina, a API informa que a temporada foi concluída. A rota next-season calcula a classificação final, avalia a diretoria ou o desempenho do atleta, evolui os jogadores, gera reposições e cria o calendário seguinte.

8. Implantação no Azure

O procedimento abaixo reproduz o roteiro do README.md. Os comandos pressupõem uma VM Ubuntu 22.04, acesso SSH e um repositório GitHub já publicado. Substitua os valores entre sinais de menor/maior pelos dados reais da implantação.

8.1. Provisionamento dos recursos

6. No portal do Azure, selecionar a assinatura de estudante e criar um Resource Group.
7. Criar uma VM Linux com Ubuntu Server 22.04 LTS. Registrar a região, o SKU, o usuário administrativo e o disco do sistema.
8. Configurar autenticação SSH por chave pública. Evitar login administrativo por senha.
9. Na configuração de rede, associar IP público, VNet, NIC e NSG. Restringir a porta 22 ao IP de administração e liberar a porta 80 para acesso HTTP à API.
10. Anotar os dados reais da VM: nome, Resource Group, região, SKU, IP público, tamanho/tipo do OS Disk e nome do NSG.

[INSERIR PRINT: Resource Group com os recursos do projeto.]

[INSERIR PRINT: Configuração da VM mostrando Ubuntu 22.04, região e SKU.]

[INSERIR PRINT: Visão geral da VM em execução e IP público.]

[INSERIR PRINT: Aba Discos mostrando o OS Disk.]

[INSERIR PRINT: Regras de entrada do NSG.]

8.2. Instalação do ambiente e clonagem

```
ssh azureuser@<IP_PUBLICO>

sudo apt update
sudo apt install -y python3-venv nginx git

sudo mkdir -p /opt/football-api
sudo chown "$USER":"$USER" /opt/football-api
git clone https://github.com/<USUARIO>/<REPOSITORIO>.git /opt/football-api
cd /opt/football-api
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
mkdir -p data
```

O nome de usuário SSH e o endereço do repositório acima são exemplos. O diretório de instalação adotado pelo README é /opt/football-api.

[INSERIR PRINT: Terminal SSH mostrando git clone e instalação das dependências.]

8.3. Serviço systemd

Criar o arquivo /etc/systemd/system/football-api.service. A variável DB_PATH informa onde o arquivo SQLite deve ser armazenado na VM.

```
[Unit]
Description=Football Career API
After=network.target

[Service]
User=azureuser
WorkingDirectory=/opt/football-api
Environment=DB_PATH=/opt/football-api/data/career.db
ExecStart=/opt/football-api/.venv/bin/uvicorn main:app --host 127.0.0.1 --port 8000
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
sudo systemctl daemon-reload
sudo systemctl enable --now football-api
systemctl status football-api
```

[INSERIR PRINT: systemctl status football-api mostrando active (running).]

8.4. Configuração do Nginx

Criar /etc/nginx/sites-available/football-api e configurar o Nginx como proxy reverso para o Uvicorn, que permanece escutando somente em 127.0.0.1:8000. O valor server_name _ permite acessar a API pelo IP público da VM sem domínio próprio.

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/football-api /etc/nginx/sites-enabled/football-api
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx
```

[INSERIR PRINT: nginx -t com a mensagem de configuração válida.]

> O projeto utiliza HTTP na porta 80 para a demonstração acadêmica. Não é necessário configurar domínio, certificado ou HTTPS para cumprir o escopo atual.

8.5. Backup agendado

```
sudo mkdir -p /var/backups/football-api
sudo chown azureuser:azureuser /var/backups/football-api
# no crontab do usuário de serviço:
0 2 * * * DB_PATH=/opt/football-api/data/career.db BACKUP_DIR=/var/backups/football-api BACKUP_RETENTION_DAYS=14
/opt/football-api/.venv/bin/python /opt/football-api/backup_db.py
```

O backup é gerado em um arquivo datado e validado com PRAGMA integrity_check. A restauração deve ser testada e realizada com o serviço parado.

8.6. Validação do acesso externo

```
http://<IP_PUBLICO>/health
http://<IP_PUBLICO>/docs
```

A rota /health deve retornar {"status":"ok"}. A rota /docs deve abrir a documentação interativa do FastAPI e permitir executar as chamadas sem autenticação.

[INSERIR PRINT: /docs aberto pelo IP público da VM, com a lista de endpoints.]

9. Testes e evidências

A suíte automatizada foi executada localmente com pytest. O resultado desta revisão foi de 9 testes aprovados. Os testes usam banco temporário para não alterar os dados de desenvolvimento e cobrem as rotas e regras de negócio principais. Isso comprova o comportamento no ambiente de teste local, mas não comprova o deploy público na Azure.

Teste automatizado	Resultado
GET /health e GET /clubs	Aprovado: API responde e catálogo contém oito clubes.
Isolamento entre carreiras	Aprovado: IDs e times de carreiras diferentes não se misturam.
Validação de escalação	Aprovado: escalação 3-5-2 válida aceita e distribuição incompatível rejeitada.
Mercado de transferências	Aprovado: venda de jogador comprado como agente livre retorna valor inferior ao custo de compra.
Cobertura mínima do elenco	Aprovado: não permite reduzir o número de meio-campistas abaixo do mínimo de 5.
Temporada do modo jogador	Aprovado: simula 14 rodadas e inicia a segunda temporada.
Acesso no escopo acadêmico	Aprovado: Swagger e endpoints podem ser acessados sem autenticação.
Backup SQLite	Aprovado: cópia criada, integrity_check retorna ok e os dados podem ser lidos.
Banco de origem ausente	Aprovado: backup falha com erro explícito quando o arquivo de origem não existe.

```
pip install -r requirements-dev.txt
pytest -q
# Resultado validado nesta revisão: 9 passed
```

9.1. Evidências que devem entrar no relatório final

Evidência	O que precisa aparecer no print
Swagger externo	URL/IP da VM visível e grupos de endpoints da Football Career API.
Criação de carreira técnica	POST /careers com mode MANAGER e resposta contendo clube, orçamento e confiança.

Evidência	O que precisa aparecer no print
Escalação	PUT /lineup com formação, tática e lista dos 11 titulares.
Rodada simulada	POST /advance mostrando os placares e os jogos da rodada.
Classificação	GET /standings após partidas simuladas.
Mercado	GET /market e uma compra/venda demonstrativa com os preços atualizados.
Acesso da API	Swagger acessível pelo IP público, com chamadas de teste sem autenticação.
Backup e testes	Saída do pytest -q e arquivo de backup criado/validado.
Carreira de jogador	GET /player com ficha e estatísticas do atleta.
Propostas	GET /offers e aceite de uma proposta.
Azure	VM em execução, SKU, região, OS Disk, NSG e IP público.
GitHub	Página do repositório com README e arquivos principais.

Os prints devem ser capturados no ambiente real de hospedagem. Não utilizar imagens ilustrativas ou respostas de testes locais como se fossem evidências do Azure.

10. Repositório no GitHub

A versão revisada do projeto está organizada nos seguintes arquivos:

Arquivo	Responsabilidade
main.py	Aplicação FastAPI, criação do esquema SQLite, regras das partidas e temporadas, rotas e operações de mercado.
requirements.txt	Dependências de execução: FastAPI e Uvicorn, com versões fixadas.
requirements-dev.txt	Dependências de desenvolvimento: httpx e pytest, além dos pacotes de execução.
tests/test_api.py	Nove testes automatizados com banco temporário.
backup_db.py	Gera backup consistente, verifica integridade e aplica retenção.
README.md	Regras, endpoints, execução local, testes, backup e roteiro de deploy Azure por HTTP.
.gitignore	Ignora ambientes virtuais, cache, banco, backups, .env e artefatos de teste.

Link do repositório: [INSERIR URL REAL DO GITHUB]

[INSERIR PRINT: página inicial do GitHub mostrando nome do repositório, README e arquivos.]

Antes de enviar o link ao professor, confirme que o repositório está público ou que o avaliador possui acesso. Verifique também se nenhum arquivo de banco com dados locais foi enviado por engano.

11. Revisão técnica e alterações realizadas

A revisão não ficou apenas no levantamento dos riscos: foram aplicadas correções no código e criados artefatos complementares. A suíte automatizada foi executada após as alterações e os nove casos passaram localmente.

11.1. Alterações implementadas

Ponto encontrado	Correção aplicada	Validação
Acesso simplificado para a entrega acadêmica.	Mantidas as rotas abertas, sem autenticação, para facilitar avaliação pelo Swagger/Postman. O README registra que a API deve ser usada com dados fictícios.	Teste confirma acesso a /clubs e /docs sem autenticação.

Ponto encontrado	Correção aplicada	Validação
Arbitragem de preço de jogador livre.	Compra de agente livre passou de 40% para 90% do valor-base; venda passou de 90% para 70%.	Teste confirma que a revenda imediata gera valor inferior ao custo de compra.
Elenco podia perder posições exigidas por formações.	Mínimos ajustados para GK≥1, DEF≥4, MID≥5 e ATT≥3.	Teste impede uma segunda venda de meio-campista quando o clube chega ao mínimo.
Escalação automática podia não respeitar a formação.	Escalação automática e manual agora passam pela mesma validação de 11 jogadores e distribuição posicional antes de gravar.	Teste aceita 3-5-2 válida e rejeita uma lista incompatível.
Ausência de testes.	Adicionada suíte pytest com TestClient/httpx e banco temporário.	Resultado desta revisão: 9 passed.
Ausência de backup reproduzível.	Criado backup_db.py com API online de backup do SQLite, verificação integrity_check e retenção padrão de 14 dias.	Teste cria, valida e lê o backup restaurado.
Regra de demissão podia ser descrita incorretamente.	README e documentação agora indicam demissão com confiança igual ou inferior a 15.	Conferido com FIRE_LIMIT = 15 e condição <= no código.

11.2. Configurações ainda dependentes do ambiente

Item	Situação real nesta entrega	Próxima ação
Azure	Os arquivos recebidos não comprovam VM, SKU, região, NSG, OS Disk ou IP real.	Preencher com dados do portal e inserir prints reais.
GitHub	URL do repositório não foi fornecida.	Inserir URL real e confirmar que os novos arquivos foram enviados.
Deploy público	O roteiro de implantação está documentado, mas os arquivos não comprovam que a API já está acessível externamente.	Executar o deploy, testar /health e /docs e capturar as evidências.
Migrações do banco	Ainda não existe ferramenta de migração de esquema.	Versionar futuras mudanças com Alembic ou estratégia equivalente, caso necessário.

11.3. Comandos para repetir a validação

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
pytest -q
```

A execução dos testes usa um banco temporário e não valida a conexão real com Azure, Nginx ou IP público. Esses itens precisam ser demonstrados por prints da infraestrutura implantada.

12. Segurança, limitações e melhorias futuras

12.1. Cuidados de segurança

- Restringir a porta SSH (22) ao IP de administração e utilizar chave SSH.
- Manter Uvicorn em 127.0.0.1:8000; expor somente o Nginx para receber HTTP na porta 80.
- Manter Ubuntu e dependências atualizados e remover serviços/portas que não são necessários.
- Não versionar .venv, cache, diretório de dados ou arquivos .db.

12.2. Limitações técnicas

- SQLite é adequado para uma API pequena, mas permite apenas uma escrita por vez por arquivo e pode limitar concorrência elevada.
- O banco está acoplado ao disco da VM; várias instâncias da API não devem compartilhar diretamente o mesmo arquivo SQLite como se fosse um banco distribuído.
- O projeto entregue é um backend REST; não foi enviado frontend gráfico de jogo.
- A API não possui autenticação de usuários; qualquer pessoa com acesso ao serviço pode executar as rotas disponíveis. O uso deve ficar restrito à demonstração com dados fictícios.
- O ambiente acadêmico utiliza HTTP sem criptografia de transporte. HTTPS pode ser adicionado se o sistema evoluir para uso público de produção.

12.3. Evoluções possíveis

- Criar um frontend web para consumir a API e apresentar escalação, calendário, tabela, mercado e perfil do jogador.
- Adicionar autenticação com tokens e autorização para que cada usuário acesse apenas suas próprias carreiras.
- Adicionar logs estruturados, métricas, alertas e monitoramento do processo e do uso de disco.
- Agendar backup_db.py no cron, monitorar as execuções e testar restauração periodicamente.
- Adicionar GitHub Actions para testes e publicação automatizada.
- Migrar para PostgreSQL ou Azure Database for PostgreSQL se a concorrência e o volume de dados crescerem.

13. Conclusão

A Football Career API evoluiu de uma API simples de simulação de partidas para um backend de modo carreira com dois caminhos de jogo, oito clubes, elencos, calendário, simulação, estatísticas, evolução de atletas, transferências e progressão de temporadas. A implementação utiliza FastAPI e SQLite, com roteiro de hospedagem em uma VM Linux no Azure.

A revisão corrigiu a fórmula econômica de transferências, reforçou a validação de escalações, adicionou testes automatizados e criou um procedimento de backup SQLite. As rotas permanecem abertas para facilitar a avaliação acadêmica, e a implantação de referência utiliza HTTP pela porta 80 sem exigir domínio ou HTTPS. Os nove testes passaram no ambiente local. Para concluir a entrega, ainda é necessário confirmar os dados reais do Azure, inserir o link do GitHub, validar o acesso público e incluir os prints solicitados.

14. Checklist final antes de enviar

- ☐ Preencher a disciplina e confirmar nome/turma/professor.
- ☐ Inserir o link real do repositório GitHub.
- ☐ Confirmar se o repositório está público ou acessível ao professor.
- ☐ Preencher o nome do Resource Group e a região Azure.
- ☐ Confirmar SKU, vCPU, RAM, OS Disk, VNet, NSG e IP público.
- ☐ Validar systemctl status football-api e nginx -t na VM.
- ☐ Abrir /health e /docs pelo IP público via HTTP e testar os endpoints no Swagger.
- ☐ Executar no Swagger pelo menos uma carreira MANAGER e uma PLAYER.
- ☐ Executar pytest -q na VM ou ambiente de desenvolvimento e capturar o resultado.
- ☐ Executar backup_db.py, validar o arquivo e documentar o agendamento.
- ☐ Capturar prints reais de Azure, terminal, Swagger, mercado, classificação e GitHub.
- ☐ Inserir o link real do repositório e URL pública confirmada.